A2C-DRL: Dynamic Scheduling for Stochastic Edge–Cloud Environments Using A2C and Deep Reinforcement Learning

Jialin Lu, Jing Yang, *Member, IEEE*, Shaobo Li, Yijun Li, Wu Jiang, Jiangtian Dai, and Jianjun Hu

***Abstract*—Resource management challenges frequently manifest in systems and networks as tough online decision tasks, for which the proper solution is dependent on an understanding of the workload and environment and facilitates smooth use of mobile edge and cloud resources. Due to the geographical dispersion of resources, constrained resource capacity, unpredictable nature of tasks, and network hierarchy present in such contexts, it is difficult to efficiently schedule jobs in edge environments. Unfortunately, existing heuristic-based methods lack generality and fast adaptability and thus cannot optimally solve such problems. The advantage actor–critic (A2C) method, on the one hand, can quickly adapt to dynamic circumstances based on relatively few data, and deep reinforcement learning (DRL) agents can on the other hand rapidly learn from their experience of environmental interactions to make better judgments. Therefore, we present an A2C-DRL real-time task scheduling technique for stochastic edge–cloud environments that enables decentralized learning and simultaneous work scheduling across multiple servers. With the aim of producing efficient scheduling decisions, we develop reward values for various resources and model the update policy, server resource scheduling method, and policy learning method. The model is adaptive and includes various hyperparameters that can be adjusted in accordance with the application requirements. We evaluate the load-balancing capability of the model by introducing a load-balancing factor. Experiments on real data sets show that the proposed A2C-DRL method outperforms seven state-of-the-art algorithms in terms of the reward value, task rejection, and the load-balancing factor.**

***Index Terms*—Deep reinforcement learning (DRL), edge computing, load balancing, resource scheduling.**

I. Introduction

RESOURCE management issues are ubiquitous in computer systems and networks. For example, problems of job scheduling in computing clusters [1], dynamically adaptive video streaming [2], and traffic control [3] which can be effectively solved by means of well-designed heuristics. However, in the current Internet of Things (IoT) era, the volumes of data are collected and processed are rapidly growing [4], and the traditional cloud-center computing paradigm cannot meet the computational needs associated with this data growth [5], [6]. In edge computing, massive numbers of tasks are processed at the edge of a network with minimal latency [7], [8] and high availability. A close examination of the most recent research in this field demonstrates that dynamic resource scheduling is required in this context [9], [10] because of the geographical dispersion and diversity of the resources in edge environments as well as their performance, cost, and stability features. Additionally, scheduling problems in edge environments are made more difficult by the unpredictable nature of edge–cloud environments in terms of the resources needed for job requests, task arrival times, and task execution timings [9], [11]. Therefore, efficient utilization of server resources in an edge environment [12] and dynamic scheduling of resources are beneficial for saving costs while improving the quality of service QoS for users [13]. The overall amount of server resources is fixed, and when the scheduling center distributes jobs to servers [14], a fair allocation technique is used to achieve load balancing [15] of server resource consumption. However, during scheduling, the servers' computing nodes [16] may be considerably outnumbered by the tasks that are submitted to them for processing, affecting the performance of the edge servers [17], [18]. In such cases, it is especially important and difficult to allocate resources sensibly. Given these challenges, we contend that reinforcement learning (RL) methods offer distinct advantages for the management of edge computing resources. Deep RL (DRL) models are able to continuously enhance their own performance when addressing unfamiliar tasks based on the reward values gained from executing the tasks. This type of learning enables DRL models to adapt to a wide range of complicated and dynamic environments. RL is a general term that describes how an agent learns a policy by interacting with its environment, whereas DRL is a specific subfield of RL in which deep neural networks are utilized to improve the performance of RL methods in addressing complex and high-dimensional problems. Consequently, DRL outperforms traditional RL in handling the challenges that arise in edge environments.

Manuscript received 29 June 2023; revised 1 October 2023 and 18 January 2024; accepted 8 February 2024. Date of publication 15 February 2024; date of current version 25 April 2024. This work was supported in part by the Guizhou Provincial Key Technology Research and Development Program under Grant QKHZC[2023]YB368, Grant QKH[2022]003, and Grant QKH[2021]335; in part by the National Natural Science Foundation of China under Grant 62166005; and in part by the Developing Objects and Projects of Scientific and Technological Talents in Guiyang City under Grant ZKHT[2023]48-8. *(Corresponding author: Jing Yang.)*

Jialin Lu and Jing Yang are with the State Key Laboratory of Public Big Data, Guizhou University, Guiyang 550025, China (e-mail: gs.lujl21@gzu.edu.cn; jyang23@gzu.edu.cn).

Shaobo Li is with the State Key Laboratory of Public Big Data, Guizhou University, Guiyang 550025, China.

Yijun Li, Wu Jiang, and Jiangtian Dai are with BAISHANCLOUD, Guiyang 550025, China.

Jianjun Hu is with the Department of Computer Science and Engineering, University of South Carolina, Columbia, SC 29208 USA.

Digital Object Identifier 10.1109/JIOT.2024.3366252

To overcome the aforementioned challenges, this study proposes A2C-DRL, a dynamic scheduling method for server resources based on DRL. This intelligent system begins with no prior task knowledge and receives rewards through environmental feedback after executing task scheduling. In this way, the model ultimately learns to output the optimal scheduling decisions to select the most suitable edge servers for users [19], [20]. In summary, the main contributions of this study are as follows.

1) In this research, considering the various types of resources consumed by various types of tasks, we use the advantage actor–critic (A2C) model to quickly adapt to environmental changes in order to make fair scheduling decisions in an edge computing environment. Accordingly, a scheduling model is effectively trained to choose a policy that can adapt to diverse circumstances with variable optimization targets by modeling multiple optimization scenarios.
2) Based on the current status of the server resources, a DRL-based server resource scheduling system (SRSS) model is proposed. Jobs in this model are planned in line with the optimization goal and the current environmental conditions to maximize the use of scarce edge resources. Additionally, the underlying algorithm uses dynamic task scheduling to fully utilize idle resources in a stochastic environment, which is crucial for efficient resource utilization and cost savings.
3) We design an intelligent server resource scheduling algorithm with edge server resource constraints, including a resource constraint module and a server state update module (SSUM), based on server parameter information and task requirements. The A2C states, actions, and reward values are constructed in accordance with actual business requirements, and on this basis, task requests are reasonably allocated to edge servers through the A2C method.
4) A library of scheduling algorithms is built, and many sets of tests are run in complex SRSS scenarios to illustrate the superiority of the proposed A2C-DRL method from various perspectives. Seven computational strategies are considered for comparison in the experiments: a) the actor–critic algorithm (AC); b) the deep Q-network algorithm (DQN); c) the double DQN algorithm (DDQN); d) Dueling DQN; e) a Monte Carlo-based policy gradient algorithm (REINFORCE); f) a randomized scheduling algorithm (RANDOM); and g) a polling scheduling algorithm (RR). The approach presented in this study leads to better results in a number of ways.

The remainder of this article is structured as follows. The literature on heuristic and RL-based load-balancing techniques is briefly reviewed in Section II. The system model and problem description are introduced in Section III. The proposed model is described in Section IV. The experimental portion of this study is discussed in Section V. This article is concluded in Section VI.

II. Related Work

To achieve server resource load balancing, computationally intensive jobs are assigned to edge servers with idle resources via a scheduling center. The basic concept in this study is to design a reward value based on server resource utilization. As background, this section summarizes the status of research on heuristic load-balancing methods and load-balancing methods based on RL.

A. Heuristic-Based Load Balancing

Various heuristic techniques have been developed to address the cloud task scheduling problem for NPs and the load-balancing problem during resource scheduling. To address the problem of increasing application demand causing servers to become overloaded or unbalanced, thus affecting the performance of cloud systems, [21] proposed a new load-balancing mechanism called load-balancing resource clustering (LB-RC). This method uses a metaheuristic bat algorithm to obtain the best source clustering with fast convergence. A dynamic task allocation strategy was also proposed to achieve the minimum manufacturing time and execution cost. To address the problem of flow collisions occurring in BSRS and the resulting wasted BCube link capacity, [22] designed single-path and multipath centralized dynamic parallel scheduling algorithms, CDPFS and CDPFSMP, respectively, and the experimental results demonstrated the robustness of these algorithms for improving the load balancing of data traffic. To address the load-balancing problem in cloud environments for mobile users, [23] proposed a resource-aware packet-level scheduling algorithm with load balancing (RPS-LB), which improves the user request processing rate in a cloud environment. To solve the problems of imbalance and low resource utilization in previous task scheduling optimization strategies, [24] proposed a strategy using an improved ant colony algorithm for use in cloud computing. This algorithm uses a task satisfaction function constructed to find the optimal task scheduling solution based on three aspects: 1) the minimum waiting time; 2) the degree of resource load balance; and 3) the task completion cost. Load balancing for virtual machines (VMs) is ensured by introducing VM load weighting factors into the local information update process. To solve the task scheduling problem for heterogeneous resources while maintaining load balancing between different resources, [25] proposed a fast heuristic algorithm based on a zero-balance approach. The method focuses on minimizing the completion time differences between heterogeneous VMs. Two quantities, the optimal completion time and the earliest completion time, are defined to consider the transmission times of VM tasks over the network bandwidth, thus effectively implementing load balancing and task scheduling. For the problem of robust DMPC for multibit uncertain systems based on elastic DOF under RR scheduling. Wang et al. [26] introduced RR scheduling mechanism to equalize the limited communication resources of distributed systems. They successfully reduced the computational complexity by decomposing the global system into

multiple subsystems. By scheduling the sensor nodes, the transmission load is reduced by utilizing RR scheduling. A new distributed performance metric is also proposed for the different couplings that exist between the subsystems, which takes into account the neighbor information of the previous iteration. To address the problem of scheduling jobs with multiple resource requirements (CPU, memory, and disk) on a distributed server platform, [27] proposed a randomized scheduling algorithm for placing jobs on servers, which is based on randomly exploring possible combinations of jobs and servers and retaining better combinations with a higher probability. In this algorithm, each queue and each server is assumed to perform only a constant number of operations in each time unit. In this way, the maximum throughput can be achieved with less complexity.

Traditional heuristic-based algorithms for solving load-balancing problems typically rely on the operator's perspective and short-term knowledge of the traffic conditions and network environment. To a certain extent, they are able to solve load-balancing problems during the task scheduling process. However, they do not consider the heterogeneity of resources or the dynamic nature of the network in an edge environment, which may be highly complex and unbalanced.

B. Reinforcement Learning-Based Load Balancing

To effectively solve the problems faced by traditional heuristic algorithms, such as dynamic changes in the data sources and network infrastructure in an edge environment, DRL can be used. For the problem of server resource load balancing considering user preferences, [28] proposed a centralized agent-based federated cloud model with user preferences, which helps to improve the utilization of cloud resources and the load-balancing capability in edge computing. Zheng et al. [29] proposed a multiobjective task scheduling optimization algorithm based on the artificial bee colony (ABC) algorithm and the Q-learning algorithm, called the MOABCQ method, to improve the efficiency of available resource allocation. To solve resource allocation problems in mobile edge computing environments, a DRL-based intelligent resource allocation algorithm (DRLRA), which minimizes the average service time by adaptively allocating computational and network resources, was proposed in [30]. To address the problem of unbalanced resource allocation in edge environments, [31] investigated the load-balancing problem for IoT edge systems with mobile objects based on the D3QN network model and combined with a long short-term memory (LSTM) neural network. For dynamic task scheduling problems in cloud computing, [32] proposed the DDQN-TS approach for task scheduling, exploiting the adaptive capabilities of DDQN to explore the optimal task scheduling strategy. Experiments showed that DDQN-TS can guarantee a high task completion rate and effectively reduce the average task response time. To address the serious system performance problems caused by unbalanced loads on metadata servers, [33] proposed a metadata load-balancing (MDLB) mechanism based on RL. This algorithm can dynamically schedule loads in accordance with server performance and has a strong ability to handle sudden changes in data volume. Considering the importance of assessing the resilience of a power system under abnormal operating conditions for measures employed in planning and operation, [34] used the DRL technique to assess the level of power system resilience (LoR) under sequential topology attacks and faults. The framework design approach was based on DRL agents which was implemented with various algorithms: DQN, DDQN, REINFORCE (a Monte Carlo-based policy gradient algorithm), and REINFORCE with a baseline. The experimental results showed that the multiple paths present in the PS3 topology support the load requirements on the generation side. The experimental results also showed that the DDQN agent was very stable and had the highest success rate among all the agents. To address the problems of load balancing and parameter improvement in cloud networks under increasing task sizes, [35] proposed a hybrid scheduling strategy based on particle swarm optimization (PSO) and the AC algorithm. The strategy achieved improvements in various parameters, such as the makespan, resource utilization, and energy consumption for load balancing, in a continuous action space. To address the problems involved in balancing the load distribution, resource allocation, and energy consumption in cloud environments, [36] proposed an approach that utilizes multiobjective optimization and RL techniques to address the challenges of scheduling independent tasks in cloud computing. This approach employs Dueling DQN networks and the prioritized empirical replay technique to simultaneously optimize the makespan and power consumption. Experimental results validated the effectiveness and reliability of this approach, which was found to outperform other existing algorithms in terms of reducing duration, power consumption, and other performance metrics. While distributed workload queues offer significant advantages in terms of decoupling, resilience, and scaling, existing placement strategies may lead to unnecessarily long execution times and high usage costs. Staffolani et al. [37] presented an RL-based queuing (RQL) adaptive task assignment system based on an implementation of the DDQN algorithm. RLQ-enabled learning-based approaches offer significant improvements over existing baselines, and they are also capable of automatically adapting to changing workloads. To solve the problems of congestion and service delay at cloud data centers due to requests sent from edge devices, [38] proposed an RL-based task scheduling algorithm named RLFS, which achieves load balancing, reduces the average response time and energy consumption. In this algorithm, the learning agent maintains an action quality table (Q-table), which is updated after each learning session in accordance with the degree of agreement of the performed actions. The learning agent updates the values of its actions in this table based on feedback signals that were received from the environment. To address the problem of unbalanced resource utilization in cellular networks, [39] proposed a decentralized mobile load-balancing (MLB) approach based on DRL with a separate reward function. This decentralized DRL-based MLB approach can achieve a more balanced cell load distribution and better edge user performance compared to the state-of-the-art MLB approach.

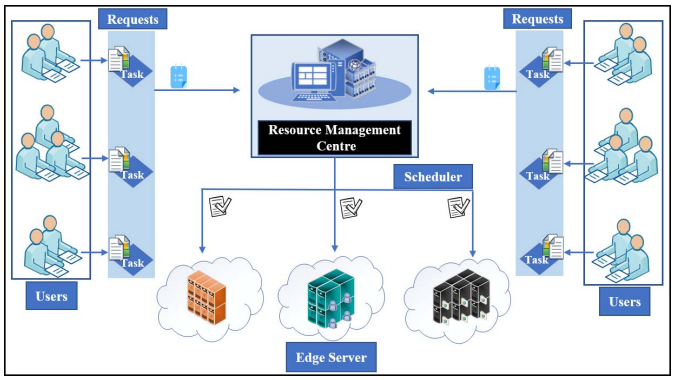


Fig. 1. Traditional edge resource scheduling process.

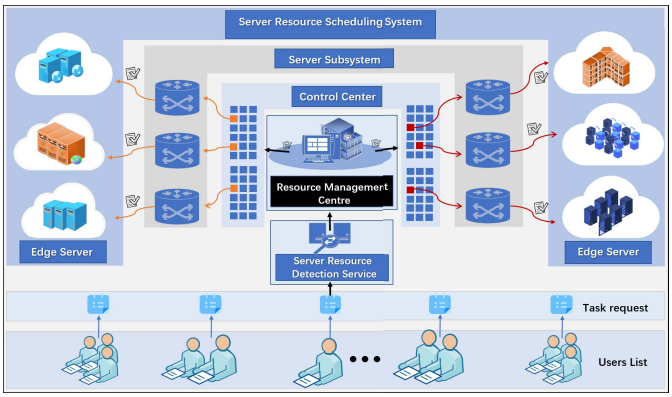


Fig. 2. Server resource scheduling system.

The above summary of previous research provides some insight into the challenges of resource load balancing [21], [22], [23], [24], [25], [26], [27], [28], [29], [30], [31], [32], [33], [34], [35], [36], [37], [38], [39], in which context efficient resource usage is a popular research topic. To this end, DRL is capable of handling complex, dynamic scheduling tasks [38]. Although all of the above studies used DRL methods to address the load-balancing problem in resource scheduling, several challenges remain, including how to provide more stable training and achieve low-variance learning. Effective utilization of server resources is critical for resource scheduling; however, previous works have not investigated reward values based on server resources. These issues need to be further addressed to improve the performance and reliability of DRL in resource scheduling tasks.

The A2C algorithm has an advantage over traditional DRL algorithms in that it usually has more stable training performance. A2C uses a simultaneous updating strategy, thus reduces the variance in training, and is therefore more likely to achieve stable learning. In addition, it can accommodate high-dimensional discrete action and state spaces or even continuous action and state spaces. Meanwhile, considering server resources, the definition of the reward value can help to better balance resource allocation and improve overall performance. Based on the above considerations, this research investigates the load-balancing task scheduling problem for server resources in edge environments and, to ensure that all server resources can be efficiently and reasonably utilized in the task scheduling process, proposes an efficient A2C-DRL algorithm that considers server resources. This method can also play an important role in improving the efficiency of utilization of scarce server resources.

III. System Modeling and Problem Description

In this section, the system model we designed is first presented, and the problem description is then introduced. Table I lists the key notations used in this article.

A. Server Resource Management in Edge Environments

Edge servers are distributed in different geographical locations to provide real-time services to users. However, task scheduling in traditional edge environments mainly relies on heuristic algorithms that do not consider the server workload or the complex nature of the resources in the edge environment. Such algorithms are unable to react to dynamic changes in the states of edge servers, resulting in insufficient server resource utilization, poor load-balancing capabilities, and idle servers, as illustrated in Fig. 1. To solve the aforementioned issues, we employ a DRL approach to create a load-balancing server resource scheduling model that improves server resource usage in an edge environment, as illustrated in Fig. 2. The SRSS depicted in Fig. 2 includes a set of users issuing requests, a scheduling control center, and a service provider. A user submits a task request to the scheduling center, which assigns an appropriate server for the task based on the current server resource status. The service provider obtains a reward by responding to the user request and feeds this reward back to the scheduling center in real time. The scheduling center module interacts with a DRL module (DRLM) to continuously optimize the scheduling policy based on the reward value. And the actions of the SRSS are decided by the scheduling center. The resource management center (RMC) manages the scheduling, resource release, and server resource detection functions that are handled by the scheduling control center's infrastructure. In the proposed SRSS (Fig. 2), the service provider is the specific implementer at the user end and connects with users who need services through the SRSS rather than through the direct load control. User requests come from different edge devices. When received, the user requests are assigned to free servers to execute the corresponding tasks through the scheduling model at the RMC. The user experience is affected by the quality of the response from the server, and when a server is overloaded with tasks, resulting in insufficient server resources, the server rejects any further user requests and stops providing new services to users.

B. Problem Description

Our primary concern is enhancing server load balancing and maximizing the performance of the scheduling center in the edge environment. The server resource scheduling center's performance is measured by the reward earned from each task execution scheduling request; the higher the reward value is, the better the implemented scheduling policy. The reward in task scheduling interval i is denoted by R_i, and the value of

TABLE I
PARAMETER INFORMATION

Symbols	Descriptions	Symbols	Descriptions
T	sequence of tasks	tr_{men}^{m}	the MEN resource needed for the nth task request
S	resource state information for the server	S_i	the resource status information of the ith server
n	the total number of task requests	$V(S_t)$	is an estimate of the value function for the current state S_t
m	number of edge servers	$V(S_{t+1})$	is an estimate of the value function for the next state S_{t+1}.
R	reward	δ	TD error value
t_{run}	the amount of time the task is running on the server	γ	discount factor
t_{cur}	current time	A_m	the task is scheduled to be executed on the mth server
t_{stu}	bias time	θ_a	parameters of the actor network
t_{dll}	deadline for the task	θ_c	parameters of the critic network
tag	the execution capacity of the server	$\bigtriangledown_{\vartheta} J(\vartheta)$	gradient with respect to the strategy parameter θ
F_{cpu}^{m}	the remaining CPU resources of the mth server	$Q(S_t, a_t) - V(S_t)$	advantage function
F_{io}^{m}	the remaining IO resources of the mth server	$loss_c$	the loss function of critic
F_{men}^{m}	the remaining MEN resources of the mth server	r_t	instant reward
tr_{cpu}^{m}	the CPU resources required for the nth task request	β	task rejection rate
tr_{io}^{m}	the IO resources required by the nth task request	σ	load balancing metrics

R_i is determined by the server load-balancing effect. Resource status of the server is denoted as S, and it includes information about the servers' CPU, I/O, and memory parameters. When a new task arrives, the scheduling center assigns it to a suitable server. The server that accepts the task is denoted by A_m, but not all task requests can be assigned; only tasks with $t \in T$ can be assigned to a server for processing. Tasks are described by triples of the form $t[tr_{\text{cpu}}, tr_{\text{io}}, tr_{\text{men}}]$, and for each task, the size of the associated server resources demand is recorded.

The proposed dynamic server resource scheduling method is designed to choose a reasonable scheduling strategy that balances the server resource loads to obtain the maximum return. The dynamic resource scheduling problem in the SRSS is defined as described in the following sections.

1) Task Request and Processing Model: The list of user requests is denoted by $T \in [tr_1, tr_2, \ldots, tr_n]$, and the set of edge servers is denoted by $ES \in [es_1, es_2, \ldots, es_m]$. The request list is a list of all task sequences submitted by users to the service provider that need to be processed, and n denotes the number of task sequences that need to be scheduled for execution in the current iteration. Edge servers are specific devices that respond to task requests, and m denotes the number of edge servers capable of providing real-time services to users.

2) Task Execution Time Model: The t_{dll} is defined as the time when task processing is completed. t_{dll} consists of three parts: 1) the time at the current moment t_{cur}; 2) the task execution time t_{run}; and 3) the transmission delay time t_{stu}, which is the time loss due to data transmission. The time when task processing is completed can be expressed as

$$t_{\text{dll}} = t_{\text{cur}} + t_{\text{run}} + t_{\text{stu}} \tag{1}$$

3) Server Resource State Update Constraint Model: After the scheduling center receives task requests from users, it performs prescheduling operations, and the status at the end of prescheduling is represented by the following tag:

$$\text{tag} = F\left[F_{\text{cpu}}^{m} + F_{\text{io}}^{m} + F_{\text{men}}^{m}\right] - T\left[tr_{\text{cpu}}^{n} + tr_{\text{io}}^{n} + tr_{\text{men}}^{n}\right] \tag{2}$$

where $F[F_{\text{cpu}}^{m} + F_{\text{io}}^{m} + F_{\text{men}}^{m}]$ represents the amounts of CPU, I/O and memory resources available at the mth server for serving user requests, while $T[tr_{\text{cpu}}^{n} + tr_{\text{io}}^{n} + tr_{\text{men}}^{n}]$ represents the amounts of CPU, IO, and memory resources required for the nth prescheduled task. If $t_{\text{cur}} + t_{\text{run}} + t_{\text{stu}} <= t_{\text{dll}}$, then

$$\text{return} = \begin{cases} \text{tag} < 0, \text{ tag } \ \forall \ < \ 0 \\ \text{tag} \geq 0, \text{ tag } \ \forall \ \geq \ 0\,. \end{cases} \tag{3}$$

When tag<0, $return$=1 meaning that there are not sufficient resources available on the server at this time, so the server cannot process the current task request. When $tag \geq 0$, $return$=0 indicating that the server has sufficient resources available, so the task request can be dispatched to this server for execution.

4) Resource Release Model: Since the edge server resources are limited, the resources occupied by tasks that have been executed need to be released in a timely manner. The latest remaining resources of a server are denoted by F. F is equal to the plus amount of resources released from the current task and the amount of resources available before the server releases the resources from the current task

$$F = F\left[F_{\text{cpu}}^{m} + F_{\text{io}}^{m} + F_{\text{men}}^{m}\right] + T\left[tr_{\text{cpu}}^{n} + tr_{\text{io}}^{n} + tr_{m}^{n}\right]. \tag{4}$$

5) Resource State Model: Only when the scheduling center performs the scheduling operation can it obtain a reward, and the reward value is determined by the resource load balance

state among the servers. Therefore, when the scheduling center performs scheduling, it needs to consider the current state S_i of all servers to make the most reasonable decision

$$S_i = \left[\left(S^1_{\text{cpu}}, S^1_{\text{io}}, S^1_{\text{men}}\right), \ldots, \left(S^m_{\text{cpu}}, S^m_{\text{io}}, S^m_{\text{men}}\right)\right]. \tag{5}$$

6) Reward Value Model: To solve the load-balancing problem among servers, we convert the degree of goodness of the server resource allocation environment into an effective reward value R

$$R = f(S_i). \tag{6}$$

This value is the evaluation value of server load balancing and is designed to reflect the load-balancing effect. The larger R is, the better the effect of load balancing and vice versa. The model assigns tasks reasonably in accordance with the state distribution of the servers. R is derived from a transformation of S_i; the specific transformation process is presented in Section IV-A. Where the R value is the evaluation value of server load balancing; the larger the R is, the better the load-balancing effect, and vice versa.

IV. Deep Reinforcement Learning and Model Updates

In this section, we introduce the RL process performed at the RMC and present a DRL-based resource scheduling paradigm. The capacity of each server under load-balanced conditions is utilized to compute the ultimate reward value for various resources. Meanwhile, the parameters of the resource scheduling model are updated by using an actor network parameter update algorithm and a critic network parameter update algorithm, which are described below.

A. Reward Value Design

Based on the current state, the DRL algorithm makes decisions, and the networks comprising the resource scheduling model are updated based on the obtained reward values. Accordingly, for the resource scheduling model to have the best effect, it should seek to obtain the maximum reward value after each task is scheduled. We let $J(\theta)$ denote the sum of the reward values obtained from the start of the state to the end of the state and let R_i denote the reward value obtained in the mth scheduling round

$$f(S_i) = \sum\nolimits_{i=0}^{m} R_i. \tag{7}$$

The DRL-based resource scheduling approach can adapt to complicated environmental changes. Different server states are used as inputs to the model, and the model outputs a decision regarding the current best action based on the input state. The reward value constrains the model to take reasonable actions and schedule tasks to appropriate servers. For this purpose, R_i is defined as the metric for updating the model parameters, and reward contributions are calculated based on the average remaining CPU, I/O, and memory resources available for the scheduling of all tasks as shown in (10).

Algorithm 1: Actor Network Parameter Update

Input: Initialize network parameters θ
Calculate the values of $V(S_{t+1})$ and $V(S_t)$
for *each t in task* **do**
  $\delta = \mathrm{r} + \gamma\ V(S_{t+1}) - V(S_t)$
  θ=*theta*+$\alpha \nabla_\theta \log \pi_\theta\ (S_t, a_t)\ \delta$
end

First, the maximum remaining available CPU, I/O, and memory resources among all servers are calculated and denoted by τ_c, τ_i, and τ_m, respectively

$$\begin{cases} \tau_c = \text{MAX}\left[\text{cpu}_1, \text{cpu}_2, \ldots, \text{cpu}_m\right] \\ \tau_i = \text{MAX}[\text{io}_1, \text{io}_2, \ldots, \text{io}_m] \\ \tau_m = \text{MAX}[\text{men}_1, \text{men}_2, \ldots, \text{men}_m] \end{cases}. \tag{8}$$

Then, we calculate the averages φ_c, φ_i, and φ_m of the remaining CPU, I/O, and memory resources, respectively, among all servers

$$\begin{cases} \varphi_c = \text{MEAN}\left[\text{cpu}_1, \text{cpu}_2, \ldots, \text{cpu}_m\right] \\ \varphi_i = \text{MEAN}[\text{io}_1, \text{io}_2, \ldots, \text{io}_m] \\ \varphi_m = \text{MEAN}[\text{men}_1, \text{men}_2, \ldots, \text{men}_m] \end{cases}. \tag{9}$$

Next, the average values of the remaining resources are converted into bonus values for different types of server resources

$$\begin{cases} \omega_c = \varphi_c/(\tau_c * n) \\ \omega_i = \varphi_c/(\tau_i * n) \\ \omega_m = \varphi_c/(\tau_m * n) \end{cases}. \tag{10}$$

Finally, the reward value obtained by the RMC is jointly calculated based on the load-balancing state of the servers and the number of rejected tasks

$$R = \omega_c + \omega_i + \omega_m. \tag{11}$$

B. Model Update Strategies

The DRLM uses the A2C algorithm, and the scheduling center's decision is obtained via the A2C-DRL algorithm, in which the decision network is divided into an actor network and a critic network. Here, "actor" refers to a policy function, and "critic" refers to a value function. The trained policy function (actor network) makes it easy to select the appropriate action from the continuous action space. The parameters of the actor network are updated in an iterative fashion, which leads to slow learning efficiency; therefore, a value function-based algorithm is also used to implement single-step updates in the critic network. Algorithm 1 is the update algorithm for actor network parameters, and Algorithm 2 is the update algorithm for critic network parameters.

C. A2C-DRL-Based Server Resource Scheduling Method

The RMC receives requests sent from user devices over the Internet, and these requests contain information about the resources needed for the requested tasks. The CPU, IO, and memory requirements of a task as well as its expected completion time and deadline affect the decision of the RMC. The RMC selects an appropriate server to execute the task

Algorithm 2: Critic Network Parameter Update

Input: Initialize network parameters θ

Calculate the values of S_t, r_t, and S_{t+1}
for *each t in task* **do**
  $\delta = \mathrm{r} + \gamma\ V(S_{t+1}) - V(S_t)$
  $\eta = \frac{1}{n} * \sum_{i=1}^{n} \delta^2$
  advantage $= V(S_{t+1}) - V(S_t)$
  return advantage
end

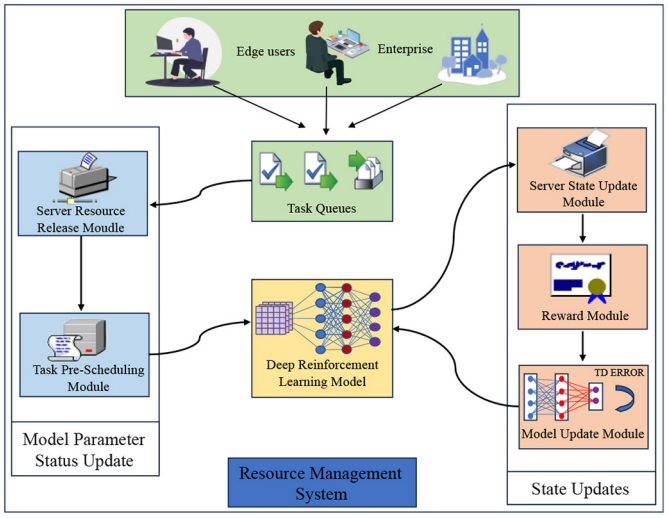


Fig. 3. SRSS.

based on the status of all servers. In the workflow of the resource scheduling management system at the RMC, as shown in Fig. 3, the task queue (TQ) contains the tasks that need to be scheduled. First, the server resource release module (SRRM) determines whether there is a task that has been executed and completed, and if there is, it releases the server resources occupied by that task. Then, the task prescheduling module (TPSM) makes a prescheduling decision and determines whether the task can be scheduled to the selected server. The DRLM outputs the scheduling decision and assigns the task to an idle server. The SSUM updates the server state based on the three model components mentioned above. The reward module (RM) calculates the reward value based on the output of the SSUM. Finally, the model update module (MUM) updates the model parameters.

Algorithm 3 describes the scheduling center's prescheduling of task requests, and the prescheduling result for a task is used to determine whether the task may be accepted by the target server and processed. The server will refuse to serve task requests for which t_{dll} cannot be satisfied.

D. Deep Reinforcement Learning

For the dynamic resource scheduling of servers in an edge environment, the current server resource state S_i is provided as the input to the actor network and the critic network, and the actor network outputs the scheduling policy π. An immediate reward value R is earned upon completion of the current

Algorithm 3: Server Resource Prescheduling

Input: Server resource status, action, task

Initialize The amounts of remaining CPU, I/O, and MEN resources at the chosen server denoted by r_{cpu}, r_{io}, and r_{men} respectively
for *each t in task* **do**
  if $t_{cur} + t_{run} <= t_{dll}$ **then**
    if $r_{cpu} >= 0$ *and* $r_{io} >= 0$ *and* $r_{men} >= 0$ **then**
      return 0
    else
      return 1
    end
  else
    return -1
  end
end

scheduling task

$$J(\theta) = \sum_{i=0}^{m} R_i \tag{12}$$

where $J(\theta)$ denotes the cumulative reward value computed from the resource states of all servers. i denotes the ith server. m denotes the total number of servers. Accumulating reward values requires accumulating multiple steps of returns, so the variance of the calculated strategy gradient is large. Therefore, A2C uses the utility function instead of cumulative rewards, which reduces variance. The gradient of the advantage function $(A(S_t, a_t))$ with respect to θ is given as

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi,\theta}\left[\nabla_\theta \log \pi_\theta(S_t, a_t) A(S_t, a_t)\right] \tag{13}$$

where θ denotes the actor network parameters and $\nabla_\theta \log \pi_\theta(s_t, a_t)$ is the gradient of the computational strategy. $A(S_t, a_t)$ represents the advantage function, which is the advantage of taking action a with respect to the average. The A2C algorithm requires training data to estimate the state value function $V(S_t)$ and the policy parameters to optimize the policy of the intelligent body. During training, the intelligent body interacts with the environment and controls the updating of the strategy gradient through the superiority function. To compute long-term payoffs, future payoffs must be taken into account, which may require multiple training sessions to stabilize the estimate of $V(S_t)$. The TD error is a method used to estimate the value function $V(S_t)$ that computes the error by comparing the estimate of the current state with the estimate of the next state. The critic network is trained to produce an estimate close to the true value function to minimize the TD error. This is one way to ensure that the value function estimate is close to the true value. The TD error (14) is represented as follows:

$$\delta = r_t + \gamma V(S_{t+1}) - V(S_t) \tag{14}$$

where δ is the TD error, which is defined as the difference between the output of the critic network and the true value of $Q(S_t, a_t)$, which is used to assess the accuracy of the critic network. The goal of critic network training is to achieve a TD error as close to zero as possible.

Algorithm 4: Dynamic Resource Scheduling Algorithm Based on A2C-DRL for Use in an Edge Environment

```
Input: Server resource status S_i
Initialize actor and critic networks
for each t in task do
    if t.status == −1 then
        updateStatus(t)
    else if t.status == 1 then
        f=actor.chooseAction(state,serverSum)
        releaseByTime(serverSum)
        rej = checkRej(f,t)
        if rej == −1 then
            t.status = −1
        if rej == 1 then
            rej_sum+ = −1
        else if rej == 0 then
            update()
            state_cur=state_next
            task.remove(t)
        end
    end
end
```

The gradient computation for the actor network using the advantage function is performed as follows:

$$\nabla_\theta J(\theta) = \nabla_\theta \log \pi_\theta(S_t, a_t)(Q(S_t, a_t) - V(S_t)). \tag{15}$$

The formula for updating the actor network based on the advantage function is

$$\theta = \theta + \alpha \nabla_\theta \log \pi_\theta(S_t, a_t)(Q(S_t, a_t) - V(S_t)). \tag{16}$$

Because the TD error provides an unbiased approximation of the dominance function, it can be used in place the of $Q(S_t, a_t) - V(S_t)$ advantage function. As a result, the formula for updating the actor network parameters becomes

$$\theta = \theta + \alpha \nabla_\theta \log \pi_\theta(S_t, a_t)\delta. \tag{17}$$

Iterative updates to the critic network are performed using the mean-square error (MSE) loss function, which is represented as

$$\begin{aligned} \text{loss}_c &= \frac{1}{n}\sum\nolimits_{n=1}^{\infty}(r_t + \gamma V(S_{t+1}) - V(S_t, \theta_c))^2 \\ &= \frac{1}{n}\sum\nolimits_{n=1}^{\infty}\delta^2 \end{aligned} \tag{18}$$

where r_t is the immediate reward, γ is a hyperparameter in the interval (0, 1), and $V(S_t, \theta_c)$ is the state value function predicted by the critic network. δ is the computed TD error value from (14).

The dynamic scheduling of edge server resources is described in Algorithm 4. The actor network generates scheduling decision actions based on the servers' resource status. The prescheduling module makes resource judgments based on the output actions. To obtain the most recent server resource status, the SSUM updates the server resources following the judgments of the prescheduling module.

V. Experiments and Analysis

In this section, we present the experimental setup, the evaluation measures, and the and compare the model proposed in this work to several benchmark algorithms for analysis. We present the experimental setup, the evaluation measures, and the comparing model in this work to the benchmark algorithm for analysis in this part.

The experiments performed in this study were carried out on a computer with an Intel Core i9-10900X CPU @3.70 GHz running a 64-bit operating system. All experiments were carried out on the same computer.

A. Experimental Setup and Experimental Data Set

An actual business environment was modeled, representing the entire process from the server receiving a task request to the task request being processed, to assess the efficacy of the method presented in this research. The execution times in an actual edge environment are described by a normal distribution since the amounts of time needed to complete various task requests in an edge environment varies. The scheduling tasks considered in this work are characterized by the quantities of daily infrastructure resources needed by the tasks and are based on data from actual servers. In the experiments, we consider a scenario with 20 edge servers delivering services to consumers, and a Markov decision method is used to allocate network resources.

Workload logs of the servers' CPU, I/O, and memory resources in the infrastructure are included in the experimental data set. The data set was chosen because it is derived from real-world business requirements and represents real-world facility utilization patterns, which are important for model training. The data set contains workload statistics for sequential timestamps (at 1-min intervals), including CPU, I/O, and memory resource usage information.

B. Evaluation Indicators

In terms of reward value, task rejection rate, and load balancing, three metrics are used to assess the robustness of the A2C-DRL scheduling center method.

1) The current scheduling action is assessed in terms of how the scheduled task request interacts with the environment during the scheduling process. The reward for the scheduling process is expressed as

$$R = \omega_c + \omega_i + \omega_m \tag{19}$$

where ω_c, ω_i, and ω_m denote the transformed reward values based on the remaining CPU, I/O, and memory resources, respectively, of the servers after the execution of the scheduled task.

2) Sometimes a task request is rejected because the server resources are occupied by other tasks and there are no free resources to serve the current task. The task rejection rate can be expressed as

$$\beta = \frac{\text{task}_{\text{reject}}}{\text{task}_{\text{sum}}} \tag{20}$$

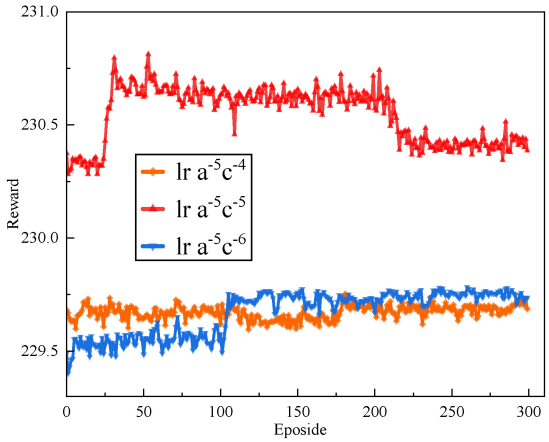


Fig. 4. Convergence results with different actor learning rates.

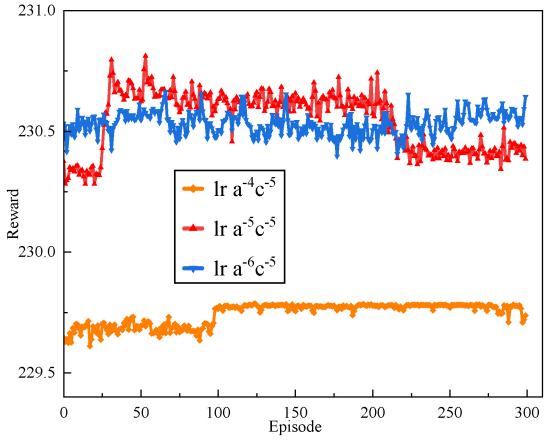


Fig. 5. Convergence results with different critic learning rates.

where $\text{task}_{\text{reject}}$ represents the number of task requests rejected by servers during the task scheduling process carried out by the scheduling center and task_{sum} represents the total number of tasks in the task sequence.

3) The load-balancing factor is determined by the degree of resource usage balance among the edge servers. The load-balancing factor is calculated as follows:

$$\sigma = \sum_{i=1}^{M} \sqrt{\frac{1}{n} \sum_{i=1}^{N} (x_i - \mu)^2} \tag{21}$$

where N represents the total number of task sequences, M represents the number of tasks to be scheduled μ represents the average remaining resources of the servers, and x_i represents the amount of resources remaining for the ith server.

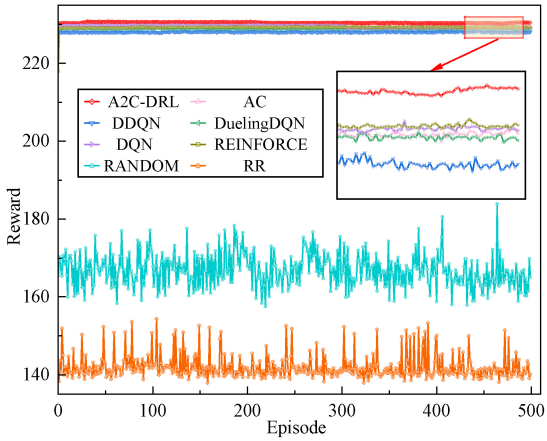


Fig. 6. Reward value.

C. Baseline Algorithms

In this study, the performance of A2C-DRL is evaluated by comparing it to seven baseline algorithms: DQN [32], [34], [37], DDQN [32], [34], [37], REINFORCE [34], AC [35], DuelingDQN [36], RANDOM [27], and RR [26]. All of the approaches discussed above are DRL methods, and the models' states are represented by S_i, as defined in (5). The action represents the transition from the current state to the next state in the state space, and the reward value is set in the same way as in all compared algorithms.

D. Convergence Analysis of the Proposed Algorithm

Figs. 4 and 5 depict the convergence of the A2C-DRL algorithm. Fig. 4 represents the convergence of the algorithm when the learning rate of the critic network is taken to be $lr = 10^{-5}$ and the learning rate of the actor network is set to different values. Fig. 5 depicts the convergence of the algorithm when the learning rate of the actor network is constant at $lr = 10^{-5}$ and different learning rates are set for the critic network. The experimental results show that the A2C-DRL method still has good convergence at different learning rates. A comparison of Figs. 4 and 5 shows that the reward values in both initially to rise very fast and eventually converge to a fixed value. The best experimental results are obtained when the learning rates of both the actor network and the critic network are set to $lr = 10^{-5}$.

E. Algorithm Performance Evaluation

The reward value, the quantity of tasks that are rejected, and the standard deviations of the remaining resources can all reflect how well the A2C-DRL scheduling algorithm makes decisions.

1) Reward Value: The performances of two conventional scheduling methods and five different DRL techniques are compared in Fig. 6. The A2C-DRL algorithm outperforms the other seven baseline algorithms in terms of the reward value. Additionally, all of the DRL methods perform better than the traditional RANDOM and RR methods. In the later stages of training, the DRL-based allocation strategies perform better than the strategies in which the environmental state is not accounted for. This is because the allocation strategy is optimized by continuously updating the actor network parameters through interaction with the environment.

2) Number of Rejected Tasks: The A2C-DRL method outperforms the other seven algorithms in terms of the number of rejected tasks, as shown in Fig. 7. In particular, the number of task rejections is less for the RANDOM algorithm than

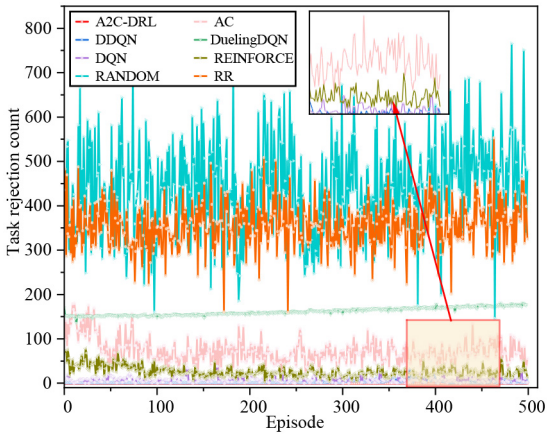


Fig. 7. Number of rejected tasks.

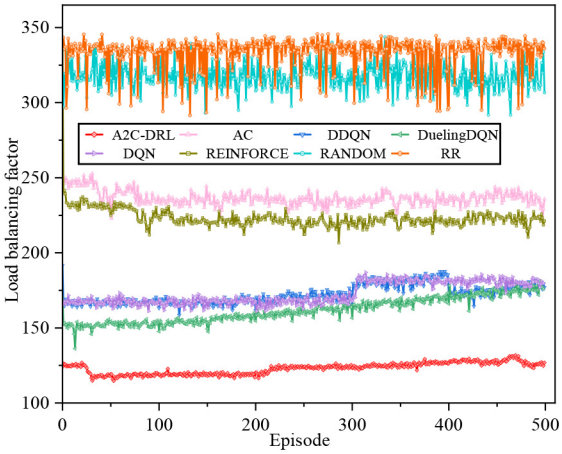


Fig. 8. Standard deviations of task counts with different numbers of servers.

for those using RL techniques. This is due to the RANDOM method's random assignment of servers to users for task completion, which ignores the servers' resource utilization. Consequently, the likelihood is high that tasks will be scheduled to run on servers whose resources are overwhelmed while leaving idle servers idle. All of the other seven algorithms have a higher number of tasks rejected after training and a wider fluctuation range compared to the A2C-DRL method. When all task sequences are scheduled, the A2C-DRL algorithm has the lowest number of task rejections, indicating that it is best able to guarantee that all tasks submitted by users are responded to a timely manner.

3) Standard Deviations of the Task Counts With Different Numbers of Servers: Fig. 8 depicts the standard deviations of the remaining resources of the edge servers during resource allocation under the various algorithms. The standard deviation of the remaining resources under the A2C-DRL algorithm proposed in this study is lower than those under the other seven methods considered for comparison, indicating that resource utilization is better balanced among the servers with the proposed method, so that each server's resources are fully utilized.

4) Standard Deviation of Different Server Tasks: The standard deviations of the task count per server with various

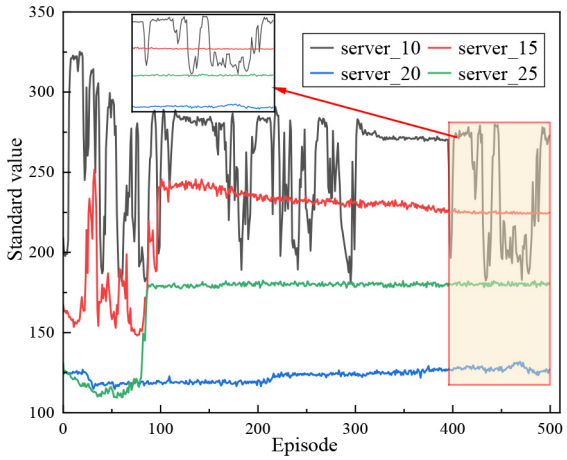


Fig. 9. Standard deviation of different server tasks.

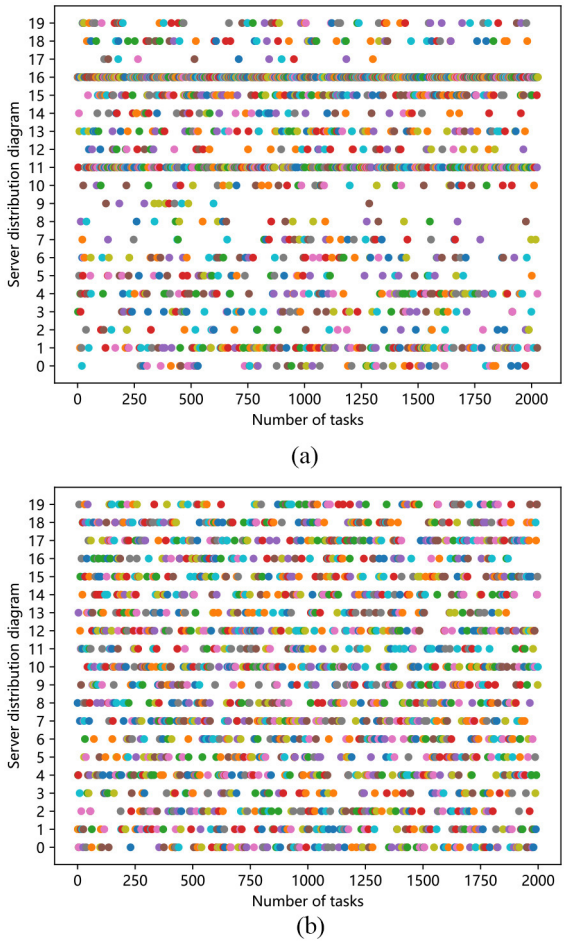


Fig. 10. Distributions of servers selected in the scheduling process. (a) Server distribution map for the REINFORCE method. (b) Server distribution map for A2C-DRL.

numbers of servers are compared in Fig. 9. According to the experimental findings, the A2C-DRL algorithm achieves the lowest standard deviation. The strategy described in this study produces the best experimental results when there are 20 servers. When there are ten servers, the standard deviation is out of balance, indicating the worst load-balancing effect.

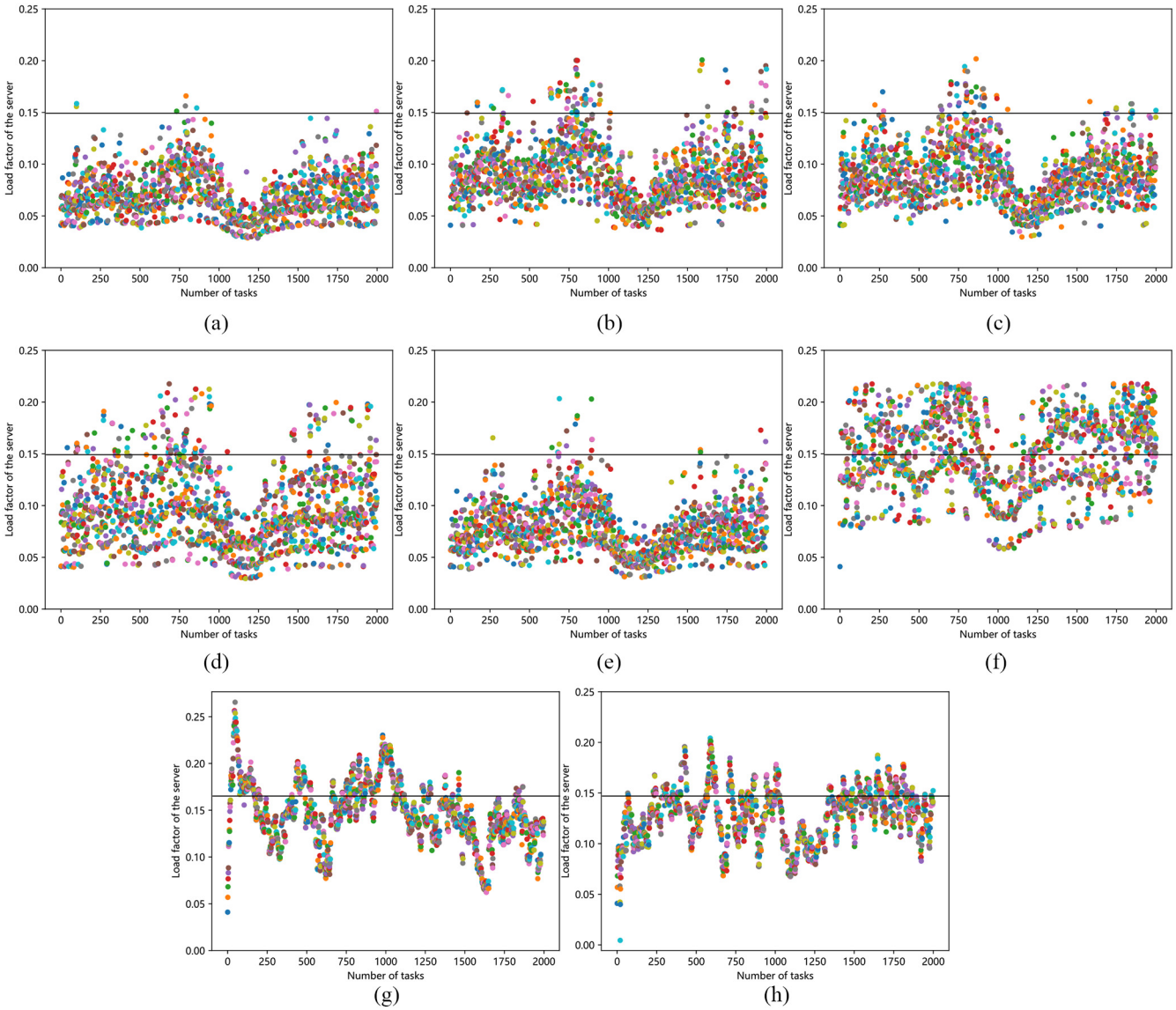


Fig. 11. Load-balancing factor. (a) A2C. (b) AC. (c) DDQN. (d) DQN. (e) Dueling DQN. (f) REINFORCE. (g) RANDOM. (h) RR.

This is primarily because there are fewer servers, which causes user requests to be delayed. Thus, the server count for the scheduling models was ultimately set to 20.

5) Server Resource Scheduling Options: Fig. 10 compares the distribution of servers selected between the REINFORCE method and the A2C-DRL method during the scheduling process. From this figure, it can be found that the load-balancing effect of the A2C-DRL method is much better than that of the REINFORCE method, with the selection of servers being more even. Free server resources are more fully utilized, and the waste of free resources is reduced. The A2C-DRL algorithm proposed in this study enables the scheduling center to continuously optimize its own scheduling strategy based on the scoring of its actions by the environment so that tasks are more evenly distributed among servers, and user requests are more reasonably distributed.

6) Load-Balancing Factor: The load factors of the various algorithms are compared in Fig. 11. The load factor of A2C-DRL is uniform, with the lowest average value. This indicates that A2C-DRL is more effective in fully utilizing all server resources during the whole scheduling process. The traditional RANDOM and RR methods have larger load factor values because these two algorithms do not consider server resource usage during scheduling.

VI. Conclusion and Future Work

This work presents the A2C-DRL resource scheduling strategy for using in edge environments to address the problem of edge server resource scheduling imbalance. Load balancing is achieved by using DRL for server resource scheduling, and the robustness of the proposed model is verified by experiments. Thus, the problem of the balanced resource allocation at the edge can be solved by using the method described in this study. Some of the main conclusions of the study are as follows.

1) According to the experimental findings, the A2C-DRL algorithm proposed in this study is effective in reducing the servers' rejection of user requests during task

scheduling. The best effect is achieved when the learning rates of both the actor and critic networks are set to 10^{-5}.

2) The optimized model is compared to the AC, DDQN, and REINFORCE models in a validation on a real data set, in which it shows a superior server task rejection rate of less than 0.1 %.
3) In the presented validation of the A2C-DRL technique on a real data set, its load-balancing factor is 0.19–0.443 lower than those of other methods, and it is found that performance can generally be improved by adopting an RL model.

In this study, the server resource scheduling process is modeled as a Markov decision process (MDP), and the scheduling model is solved using the A2C-DRL algorithm. In future work, we will investigate the possibility that if the scheduling model can obtain the sequence of tasks that need to be processed in advance, the model may be able to achieve better resource planning, reducing the waiting time and the load on the system. To this end, we will predict future task requests by considering the server state and use the predicted results as additional inputs to the model to achieve more efficient task scheduling.

REFERENCES

[1] R. Grandl, G. Ananthanarayanan, S. Kandula, S. Rao, and A. Akella, "Multi-resource packing for cluster schedulers," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 44, no. 4, pp. 455–466, 2014.

[2] Y. Sun et al., "CS2P: Improving video bitrate selection and adaptation with data-driven throughput prediction," in *Proc. ACM SIGCOMM Conf.*, 2016, pp. 272–285.

[3] M. Dong, Q. Li, D. Zarchy, P. B. Godfrey, and M. Schapira, "PCC: Re-architecting congestion control for consistent high performance," in *Proc. 12th USENIX Conf. Netw. Syst. Des. Implement.*, 2015, pp. 395–408. [Online]. Available: https://www.usenix.org/conference/nsdi15/technical-sessions/presentation/dong

[4] A. Mazloomi, H. Sami, J. Bentahar, H. Otrok, and A. Mourad, "Reinforcement learning framework for server placement and workload allocation in multiaccess edge computing," *IEEE Internet Things J.*, vol. 10, no. 2, pp. 1376–1390, Jan. 2023.

[5] Q. Luo, S. Hu, C. Li, G. Li, and W. Shi, "Resource scheduling in edge computing: A survey," *IEEE Commun. Surveys Tuts.*, vol. 23, no. 4, pp. 2131–2165, 4th Quart., 2021.

[6] S. Liu, Q. Yang, S. Zhang, T. Wang, and N. N. Xiong, "MIDP: An MDP-based intelligent big data processing scheme for vehicular edge computing," *J. Parallel Distrib. Comput.*, vol. 167, pp. 1–17, Sep. 2022.

[7] J. Wang, K. Liu, B. Li, T. Liu, R. Li, and Z. Han, "Delay-sensitive multi-period computation offloading with reliability guarantees in fog networks," *IEEE Trans. Mobile Comput.*, vol. 19, no. 9, pp. 2062–2075, Sep. 2020.

[8] T. Zhu, Z. Cai, X. Fang, J. Luo, and M. Yang, "Correlation aware scheduling for edge-enabled Industrial Internet of Things," *IEEE Trans. Ind. Informat.*, vol. 18, no. 11, pp. 7967–7976, Nov. 2022.

[9] S. Tuli, S. Ilager, K. Ramamohanarao, and R. Buyya, "Dynamic scheduling for stochastic edge-cloud computing environments using A3C learning and residual recurrent neural networks," *IEEE Trans. mobile Comput.*, vol. 21, no. 3, pp. 940–954, Mar. 2020.

[10] M. Chen, T. Wang, K. Ota, M. Dong, M. Zhao, and A. Liu, "Intelligent resource allocation management for vehicles network: An A3C learning approach," *Comput. Commun.*, vol. 151, pp. 485–494, Feb. 2020.

[11] G. Zhou, R. Wen, W. Tian, and R. Buyya, "Deep reinforcement learning-based algorithms selectors for the resource scheduling in hierarchical cloud computing," *J. Netw. Comput. Appl.*, vol. 208, Dec. 2022, Art. no. 103520.

[12] Z. Sharif, L. T. Jung, I. Razzak, and M. Alazab, "Adaptive and priority-based resource allocation for efficient resources utilization in mobile-edge computing," *IEEE Internet Things J.*, vol. 10, no. 4, pp. 3079–3093, Feb. 2023.

[13] A. U. Rehman et al., "Dynamic energy efficient resource allocation strategy for load balancing in fog environment," *IEEE Access*, vol. 8, pp. 199829–199839, 2020.

[14] C. Wang et al., "Joint server assignment and resource management for edge-based MAR system," *IEEE/ACM Trans. Netw.*, vol. 28, no. 5, pp. 2378–2391, Oct. 2020.

[15] A. Sarah, G. Nencioni, and M. M. I. Khan, "Resource allocation in multi-access edge computing for 5G-and-beyond networks," *Comput. Netw.*, vol. 227, May 2023, Art. no. 109720.

[16] B. Cao et al., "Large-scale many-objective deployment optimization of edge servers," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 6, pp. 3841–3849, Jun. 2021.

[17] Y. Mao, C. You, J. Zhang, K. Huang, and K. B. Letaief, "A survey on mobile edge computing: The communication perspective," *IEEE Commun. Surveys Tuts.*, vol. 19, no. 4, pp. 2322–2358, 4th Quart., 2017.

[18] J. Xu, L. Chen, and P. Zhou, "Joint service caching and task offloading for mobile edge computing in dense networks," in *Proc. IEEE Conf. Comput. Commun.*, 2018, pp. 207–215.

[19] F. Sun, X. Kong, J. Wu, B. Gao, K. Chen, and N. Lu, "DSM pricing method based on A3C and LSTM under cloud-edge environment," *Appl. Energy*, vol. 315, Jun. 2022, Art. no. 118853.

[20] J. Du et al., "Resource pricing and allocation in MEC enabled blockchain systems: An A3C deep reinforcement learning approach," *IEEE Trans. Netw. Sci. Eng.*, vol. 9, no. 1, pp. 33–44, Jan./Feb. 2022.

[21] M. Adhikari, S. Nandy, and T. Amgoth, "Meta heuristic-based task deployment mechanism for load balancing in IaaS cloud," *J. Netw. Comput. Appl.*, vol. 128, pp. 64–77, Feb. 2019.

[22] W.-K. Chung, Y. Li, C.-H. Ke, S.-Y. Hsieh, A. Zomaya, and R. Buyya, "Dynamic parallel flow algorithms with centralized scheduling for load balancing in cloud data center networks," *IEEE Trans. Cloud Comput.*, vol. 11, no. 1, pp. 1050–1064, Jan.–Mar. 2023.

[23] V. D. Chakravarthy and B. Amutha, "Software-defined network assisted packet scheduling method for load balancing in mobile user concentrated cloud," *Comput. Commun.*, vol. 150, pp. 144–149, Jan. 2020.

[24] X. Wei, "Task scheduling optimization strategy using improved ant colony optimization algorithm in cloud computing," *J. Ambient Intell. Hum. Comput.*, vol. 750, pp. 1–12, Oct. 2020.

[25] L. Kong, J. P. B. Mapetu, and Z. Chen, "Heuristic load balancing based zero imbalance mechanism in cloud computing," *J. Grid Comput.*, vol. 18, no. 1, pp. 123–148, 2020.

[26] J. Wang, Y. Song, G. Wei, and Y. Dong, "Resilient distributed MPC for systems under synchronous round-robin scheduling," *J. Frankl. Inst.*, vol. 358, no. 3, pp. 1957–1983, 2021.

[27] K. Psychas and J. Ghaderi, "Randomized algorithms for scheduling multi-resource jobs in the cloud," *IEEE/ACM Trans. Netw.*, vol. 26, no. 5, pp. 2202–2215, Oct. 2018.

[28] H. Liu, Y. Li, and S. Wang, "Request scheduling combined with load balancing in mobile-edge computing," *IEEE Internet Things J.*, vol. 9, no. 21, pp. 20841–20852, Nov. 2022.

[29] T. Zheng, J. Wan, J. Zhang, and C. Jiang, "Deep reinforcement learning-based workload scheduling for edge computing," *J. Cloud Comput.*, vol. 11, no. 1, p. 3, 2022.

[30] J. Wang, L. Zhao, J. Liu, and N. Kato, "Smart resource allocation for mobile edge computing: A deep reinforcement learning approach," *IEEE Trans. Emerg. Topics Comput.*, vol. 9, no. 3, pp. 1529–1541, Jul.–Sep. 2021.

[31] Q. Liu, T. Xia, L. Cheng, M. Van Eijk, T. Ozcelebi, and Y. Mao, "Deep reinforcement learning for load-balancing aware network control in IoT edge systems," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 6, pp. 1491–1502, Jun. 2022.

[32] Z. Tong, F. Ye, B. Liu, J. Cai, and J. Mei, "DDQN-TS: A novel bi-objective intelligent scheduling algorithm in the cloud environment," *Neurocomputing*, vol. 455, pp. 419–430, Sep. 2021.

[33] Z.-Q. Wu, J. Wei, F. Zhang, W. Guo, and G.-W. Xie, "MDLB: A metadata dynamic load balancing mechanism based on reinforcement learning," *Front. Inf. Technol. Electron. Eng.*, vol. 21, no. 7, pp. 1034–1046, 2020.

[34] M. Ibrahim, A. Alsheikh, and R. Elhafiz, "Resiliency assessment of power systems using deep reinforcement learning," *Comput. Intell. Neurosci.*, vol. 2022, Apr. 2022, Art. no. 2017366.

[35] A. Pradhan, S. K. Bisoy, and M. Sain, "Action-based load balancing technique in cloud network using actor-critic-swarm optimization," *Wireless Commun. Mobile Comput.*, vol. 2022, Jun. 2022, Art. no. 6456242.

[36] A. Chraibi, S. Ben Alla, A. Touhafi, and A. Ezzati, "A novel dynamic multi-objective task scheduling optimization based on Dueling DQN and PER," *J. Supercomput.*, vol. 79, pp. 21368–21423, Jun. 2023.

[37] A. Staffolani, V.-A. Darvariu, P. Bellavista, and M. Musolesi, "RLQ: Workload allocation with reinforcement learning in distributed queues," *IEEE Trans. Parallel Distrib. Syst.*, vol. 34, no. 3, pp. 856–868, Mar. 2023.
[38] F. Ramezani Shahidani, A. Ghasemi, A. Toroghi Haghighat, and A. Keshavarzi, "Task scheduling in edge-fog-cloud architecture: A multi-objective load balancing approach using reinforcement learning algorithm," *Computing*, vol. 105, no. 6, pp. 1337–1359, Jan. 2023.
[39] H.-H. Chang, H. Chen, J. Zhang, and L. Liu, "Decentralized deep reinforcement learning meets mobility load balancing," *IEEE/ACM Trans. Netw.*, vol. 31, no. 2, pp. 473–484, Apr. 2023.

Jialin Lu received the B.E. degree from Jiangxi Agricultural University, Nanchang, China, in 2021. He is currently pursuing the M.S. degree with the State Key Laboratory of Public Big Data, Guizhou University, Guiyang, China.

His research interests include edge computing and resource scheduling.

Jing Yang (Member, IEEE) received the Ph.D. degree in mechanical and electronic engineering from Guizhou University, Guiyang, China, in 2020.

From August 2018 to September 2019, he was awarded a scholarship by the China Scholarship Council under the State Scholarship Fund to pursue his study with Oklahoma State University, as a joint Ph.D. student with the Institute for Mechatronic Engineering, where he joined the Guoliang Fan's Group. From October 2022 to October 2023, he was a Visiting Scholar studying in the team of Prof. Wu Fan (NSFC Excellent Young Scholars) with Shanghai Jiao Tong University, Shanghai, China. He is currently an Assistant Professor with the State Key Laboratory of Public Big Data, Guizhou University. His research interests include open domain visual learning, intelligent robot, and edge computing. He has published more than 50 peer-reviewed papers in the related area, including well-archived international journals, such as the *International Journal of Intelligent Systems*, the *Micromachines*, the *Journal of Sensors*, and the *International Journal of Advanced Robotic Systems*.

Dr. Yang has served on the editorial board of the IEEE TRANSACTIONS ON SEMICONDUCTOR MANUFACTURING and the *Journal of Computational Design and Engineering* and a peer reviewer for a number of international conferences and journals.

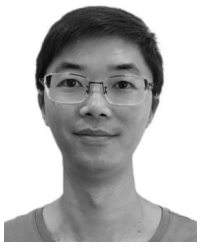
Shaobo Li received the Ph.D. degree in computer software and theory from the Chinese Academy of Sciences, Beijing, China, in 2003.

He is currently a Professor with the School of Mechanical Engineering, Guizhou University, Guiyang, China, and also serves as a part-time Doctoral Tutor with the Chinese Academy of Sciences. He is also with the Key Laboratory of Advanced Manufacturing Technology, Ministry of Education, Guizhou University. He has published more than 200 articles in major journals and international conferences. His research has been supported by the National Science Foundation of China and the National High-Tech Research and Development Program (863 Program). His current research interests include big data in manufacturing and open domain visual learning.

Prof. Li has received honors and awards from New Century Excellent Talents in the University of Ministry of Education of China, an excellent expert and innovative talent of Guizhou, the Group Leader of manufacturing information, and an alliance Vice Chairman of the intelligent manufacturing industry in Guizhou.

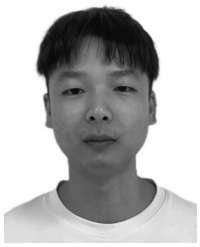
Yijun Li received the B.E. degree from the North University of China, Taiyuan, China, in 2018, where he is currently working in Baishan Cloud.

He currently serves as the Senior Director of R&D for product lines, such as CDN, security, and SD-WAN with White Mountain Cloud.



Wu Jiang received the B.E. degree from Huaqiao University, Quanzhou, China, in 2015, where he is currently working in Baishan Cloud.

He is currently with Baishan Cloud focusing on the areas of edge cloud native and new resources for fog distribution.



Jiangtian Dai received the B.E. degree from Fujian Normal University, Fuzhou, Fujian, China, in 2018, where he is currently working in Baishan Cloud.

He is currently responsible for CDN quality operations with Baishan Cloud.

Jianjun Hu received the B.S. and M.S. degrees in mechanical engineering from Wuhan University of Technology, Wuhan, China, in 1995 and 1998, respectively, and the Ph.D. degree in computer science from Michigan State University, East Lansing, MI, USA, in 2004, with a focus on machine learning and evolutionary computation.

He was a Postdoctoral Fellow of Bioinformatics with Purdue University, West Lafayette, IN, USA, and the University of Southern California, Los Angeles, CA, USA, from 2004 to 2007. He is currently an Associate Professor with the Department of Computer Science and Engineering, University of South Carolina, Columbia, SC, USA. He is also a Visiting Professor with the School of Mechanical Engineering, Guizhou University, Guiyang, China. His research interests include machine learning, deep learning, data mining, evolutionary computation, fault diagnosis, bioinformatics, and material informatics.